Handout

TF-US-PSS

Softwareanforderungen / Anwendungsfalldiagramm

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

WBS TRAINING

Inhalt

1 Softwareanforderungen 2
1.1 Softwareanforderungen: Das Fundament erfolgreicher Entwicklung 2
1.2 Struktur & Visualisierung 3
2 UML 4
2.1 Was ist die UML? 4
3 Anwendungsfalldiagramm 6
3.1 Übersicht 6
3.1.1 Erklärung / Definition 6
3.2 Syntax 7
3.2.1 Anwendungsfall und Akteur 7
3.2.2 Assoziation 8
3.2.3 Generalisierung & Spezialisierung 9
3.2.4 Include 10
3.2.5 Extend 11
3.2.6 Syntax Zusammenfassung 12
3.3 Übungsaufgabe 12
3.3.1 Aufgabenstellung 12
3.3.2 Lösung 13

Dieses Handout wurde mit Unterstützung von KI erstellt.

1 Softwareanforderungen

1.1 Softwareanforderungen: Das Fundament erfolgreicher Entwicklung

Die **Softwareanforderungen** bilden die unverzichtbare Basis für jedes Entwicklungsprojekt. Sie dienen als klare Spezifikation und gewährleisten, dass das finale Produkt exakt die Erwartungen der Stakeholder erfüllt.

Die Bedeutung von Anforderungen

Der **Grundsatz** in der Softwareentwicklung lautet: **Bevor der Code fließt, muss die Funktion klar sein.** Ein frühes, umfassendes Verständnis der Anforderungen ist entscheidend, um Fehlentwicklungen, kostspielige Nacharbeiten und Missverständnisse zu vermeiden. Ohne klar definierte Anforderungen gleicht die Entwicklung dem Bau eines Hauses ohne Blaupause – das Ergebnis ist dem Zufall überlassen.

Definition und Ziel

Anforderungen sind die **detaillierte Beschreibung** der durch das System zu erbringenden **Dienstleistungen** (*Was* das System tun soll) sowie aller damit verbundenen **Betriebs-Einschränkungen** (*Wie* das System funktionieren muss).

Das **Ziel** der Anforderungsdefinition ist es, **genau festzulegen**, **was** die Software tun soll und **wie** sie unter spezifischen Bedingungen **funktionieren muss**. Dies beinhaltet in der Regel:

- **Funktionale Anforderungen (Das WAS)**
    - Die funktionalen Anforderungen sind das Herzstück. Sie beschreiben was die Software tun soll – das konkrete Verhalten und die bereitgestellten Funktionen. Sie sind direkt darauf ausgerichtet, welche Aufgaben der Nutzer mit dem System erledigen können muss.
    - Denken Sie an das Beispiel: 'Der Benutzer muss sich mit E-Mail-Adresse und Passwort anmelden können.' Das ist eine Funktion.
    - Oder: 'Das System muss den aktuellen Lagerbestand anzeigen.' Auch das ist eine Funktion. Diese Anforderungen sind das Fundament; sie definieren den Zweck der Software.

- **Nicht-funktionale Anforderungen (Das WIE)**
    - Die nicht-funktionalen Anforderungen definieren, wie die Software funktionieren soll. Sie sind die Qualitätsmerkmale und Einschränkungen, die das Benutzererlebnis maßgeblich beeinflussen.
    - Hier sprechen wir über Dinge wie Performance: 'Die Anmeldeseite muss innerhalb von 2 Sekunden geladen sein.' Das ist nicht *was* die Seite tut, sondern *wie schnell* sie es tut.
    - Oder Zuverlässigkeit: 'Das System muss eine 99,9 % Verfügbarkeit aufweisen.' Es geht um die Qualität und Stabilität des Betriebs.
    - Die sorgfältige Erfassung und Dokumentation dieser Anforderungen ist der erste und wichtigste Schritt auf dem Weg zu einer qualitativ hochwertigen Softwarelösung.
- **Zusammenfassung**
    - Merken Sie sich bitte: Die **klaren funktionalen Anforderungen** bilden das Fundament – sie sind das *Was*. Die **nicht-funktionalen Anforderungen** definieren die Qualität und Akzeptanz der Software – sie sind das *Wie*. Nur wenn wir beides von Beginn an präzise und vollständig erfassen, können wir am Ende eine Software liefern, die nicht nur funktioniert, sondern auch begeistert."

1.2 Struktur & Visualisierung

*Vom **Text** (Anforderungen) zur **Struktur** (Anwendungsfalldiagramm - Use Case)*

Textuelle Basis (Die Funktion)	Visuelle Struktur (Der Überblick)
• Funktionale Anforderungen • Beschreibt das, WAS in Textform • Lange Listen / User Stories	• Anwendungsfalldiagramm (Use Case Diagram) • Zeigt das, WAS in einer Grafik • Einfache Darstellung der Interaktion

Der Weg von den gesammelten **Anforderungen** hin zu einer klaren **Struktur** ist ein entscheidender Schritt in der Softwareentwicklung. Während Anforderungen zunächst in Textform vorliegen, bietet die Visualisierung einen unverzichtbaren Überblick über die Systemfunktionen.

Textuelle Basis (Die Funktion)

Die textuelle Basis dient als primäre Quelle für die genaue Beschreibung aller Systemleistungen und umfasst:

- **Funktionale Anforderungen:** Hier wird detailliert in Textform beschrieben, **WAS** das System leisten soll (z.B. "Das System soll eine Suche nach Kundendaten ermöglichen").
- **Formate:** Diese Informationen liegen oft in Form von **langen Listen** oder modernen **User Stories** vor.

Die textuelle Basis ist somit die detaillierte Quelle, die die *gesamte* Funktionalität des Systems dokumentiert.

isuelle Struktur (Der Überblick)

Um die Komplexität der Textdokumente zu reduzieren und einen schnellen Überblick zu ermöglichen, kommt die **Visuelle Struktur** zum Einsatz.

- **Anwendungsfalldiagramm (Use Case Diagram):** Dies ist das zentrale Werkzeug für die Visualisierung.
- **Ziel:** Es zeigt das **WAS** (die Funktionen) in einer leicht erfassbaren **Grafik** und bietet eine einfache Darstellung der **Interaktion** zwischen den Akteuren (Benutzer, andere Systeme) und dem zu entwickelndes System.

Kurz gesagt: Die **textuelle Basis** liefert das *Detail*, das **Anwendungsfalldiagramm** liefert den *Überblick* über die zentralen Funktionen.

2 UML

2.1 Was ist die UML?

Die **Unified Modeling Language (UML)** ist eine standardisierte, visuelle Modellierungssprache, die ein unverzichtbares Werkzeug in der modernen System- und Softwareentwicklung darstellt. Sie dient dazu, komplexe Systeme auf einer abstrakten Ebene zu beschreiben, zu planen und zu dokumentieren.

Definition

Die UML ist eine **standardisierte, grafische Notation** (ISO/IEC 19505), die Elemente zur Modellierung der **statischen Struktur** und des **dynamischen Verhaltens** von Systemen bereitstellt. Obwohl sie primär für die objektorientierte Softwareentwicklung konzipiert wurde, ist sie flexibel genug, um auch Geschäftsprozesse und andere Nicht-Software-Systeme zu modellieren.

2.2 Der Zweck der UML (Ihre zentralen Vorteile)

Die Anwendung der UML verfolgt vier Hauptziele und macht den Entwicklungsprozess effizienter, transparenter und qualitativ hochwertiger:

Visuelle Darstellung und Spezifikation (Die Blaupause):

Die UML bietet eine reiche Palette an Diagrammtypen (wie Klassen-, Sequenz- oder Aktivitätsdiagramme), um die verschiedenen Aspekte eines Systems klar und verständlich darzustellen.

Sie dient als formelle Spezifikation und Entwurf, wodurch Fehler und Inkonsistenzen bereits in der Planungsphase identifiziert werden können, bevor die eigentliche Programmierung beginnt.

Effektive Kommunikation:

UML-Diagramme etablieren eine **gemeinsame Sprache** für alle am Projekt Beteiligten – von Entwicklern über Designer bis hin zu Kunden und Managern.

Dies fördert das gegenseitige Verständnis und reduziert Missverständnisse, die durch vage Beschreibungen in Textform entstehen können.

Systemmodellierung und Konstruktion:

Sie ermöglicht die detaillierte Modellierung von Struktur, Verhalten und Interaktionen eines Systems und liefert eine fundierte Planungsgrundlage.

UML-Modelle können als Basis für die direkte **Codegenerierung** verwendet werden.

Dokumentation:

Die Modelle dienen als wertvolle und einheitliche Dokumentation der Softwarearchitektur. Dies ist entscheidend für die spätere Wartung, das Einarbeiten neuer Teammitglieder und die Weiterentwicklung des Systems.

Zusammenfassend lässt sich sagen, dass UML ein mächtiges Werkzeug ist, das die Softwareentwicklung effizienter, transparenter und qualitativ hochwertiger macht.

2.3 Übersicht UML-Diagramme

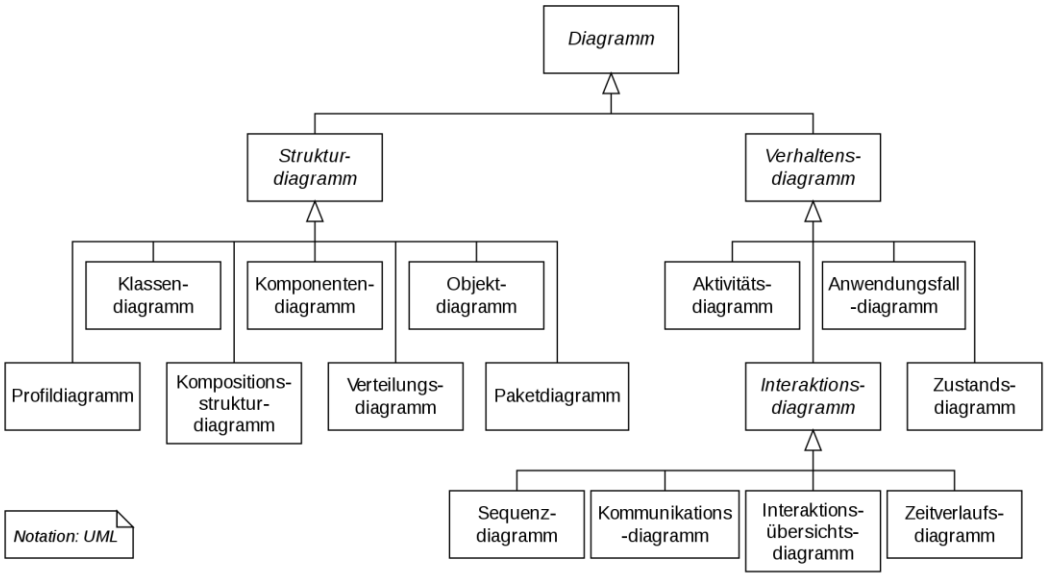


Abb.1 Übersicht der UML-Diagramme

3 Anwendungsfalldiagramm

3.1 Übersicht

3.1.1 Erklärung / Definition

Ein UML Use-Case-Diagramm, auch Anwendungsfalldiagramm genannt, ist ein Verhaltensdiagramm, das die erwarteten Funktionen eines Systems oder einer Software aus der Perspektive eines Akteurs darstellt. Dieser Akteur kann sowohl ein menschlicher Benutzer als auch ein externes System sein. Es ist wichtig zu beachten, dass ein Use-Case-Diagramm nicht die genaue Reihenfolge der Abläufe darstellt, dafür werden andere UML-Diagramme wie Aktivitäts- oder Sequenzdiagramme verwendet. Im Wesentlichen bietet ein Use-Case-Diagramm eine abstrakte Übersicht über die Funktionen und Dienste, die ein System für einen Anwender bereitstellt.

Beispiel 1:

Kunde kann Geld auszahlen lassen, dabei muss Kunde immer die PIN eingeben. Ist die PIN falsch, wird der Vorgang abgebrochen. Der Vorgang kann zu jeder Zeit auch durch Kunde abgebrochen werden. Kunde kann Geld einzahlen.

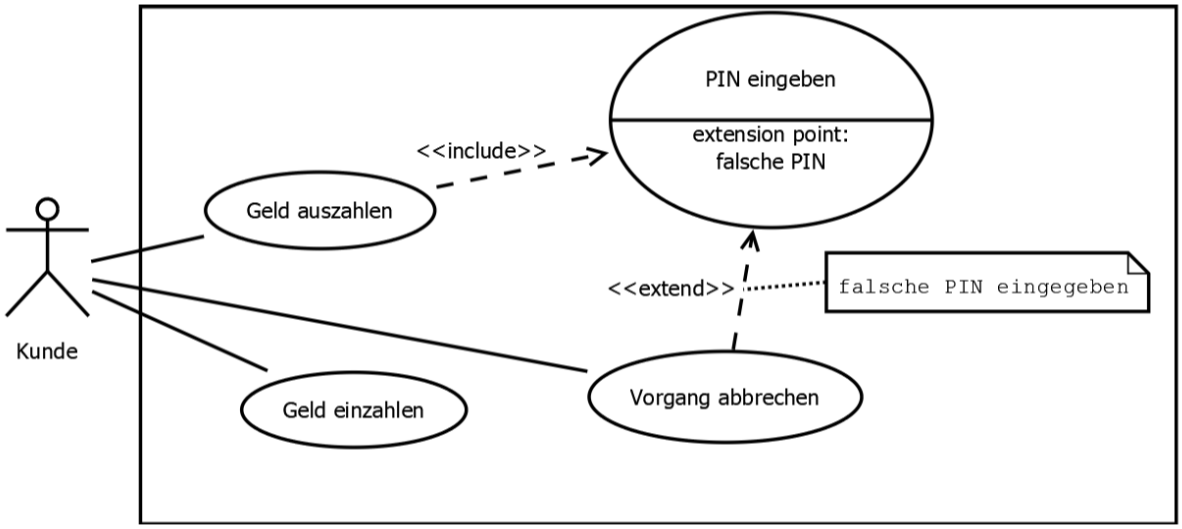
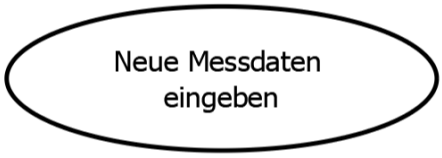


Abb.2 Darstellung Use-Case-Diagramm Kunde-Geldautomat

3.2 Syntax

3.2.1 Anwendungsfall und Akteur



Ein Anwendungsfalldiagramm besteht aus einem oder mehreren Anwendungsfällen. Anwendungsfälle beschreiben die Funktionen, die ein Akteur ausführen/auslösen/durchführen kann

Akteure sind Personen oder Systeme, die mit der Anwendung interagieren. Hierbei werden nicht direkt Menschen bezeichnet, sondern Rollen oder Personengruppen definiert.

Zum Beispiel „Buchhaltung“, oder „Benutzer“, „Admin“

Akteure stehen immer außerhalb des zu modellierenden Systems!

3.2.2 Assoziation

Akteure sind über Assoziationen mit den Anwendungsfällen verbunden.

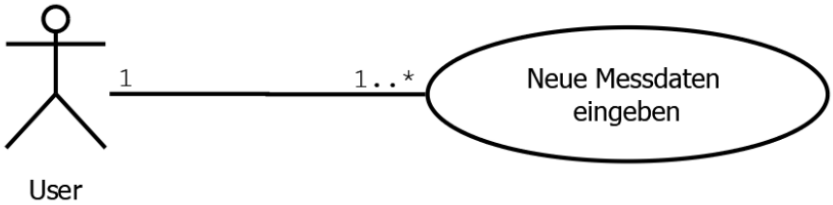


Abb.3 Darstellung Assoziation User zu Anwendungsfall „Neue Messdaten eingeben“

Diese Assoziationen können über Multiplizitäten verfügen (ein User kann 1 oder mehrere neue Messdaten eingeben, aber ein Set von Messdaten wird nur von einem User eingegeben)

Sie können auch beschriftet werden und Pfeilspitzen haben. Die Pfeilspitzen geben hier die möglichen Kommunikationsrichtungen an. (Kunde und Mitarbeiter können in beide Richtungen miteinander kommunizieren)

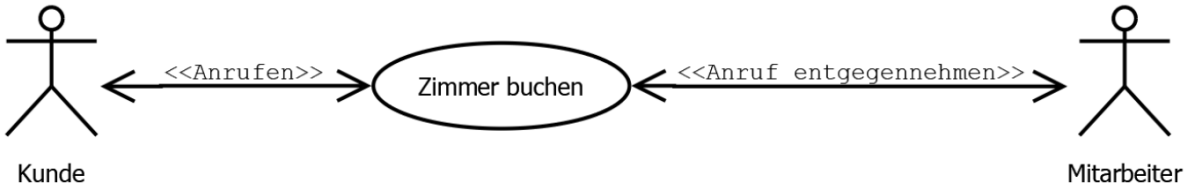


Abb.4 Darstellung zweier Akteure mit einer Assoziation zum Anwendungsfall „ Zimmer buchen“

Ist mehr als ein Akteur an einem Anwendungsfall beteiligt, müssen alle beteiligten Akteure aktiv werden, um den Anwendungsfall auszuführen. (Ein Kunde kann nicht ohne Mitarbeiter ein Zimmer buchen)

3.2.3 Generalisierung & Spezialisierung

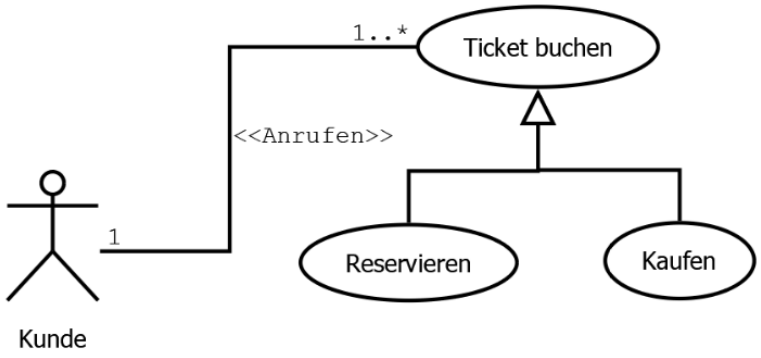


Abb.5 Darstellung Generalisierung & Spezialisierung von Anwendungsfällen

Anwendungsfälle können generalisiert und spezialisiert sein. Dies funktioniert ähnlich wie die Vererbung bei Klassen. Hier erben „Reservieren“ und „Kaufen“ alle Eigenschaften und Assoziationen von „Ticket buchen“.

Da „Kunde“ den Anwendungsfall „Ticket buchen“ ausführen kann, kann „Kunde“ auch „Reservieren“ und „Kaufen“ ausführen.

Auch Akteure können generalisiert und spezialisiert sein.

Hier erbt „Registrierter Kunde“ alle Eigenschaften und Assoziationen von „Gast-Kunde“.

Da „Gast-Kunde“ den Anwendungsfall „Reservieren“ ausführen kann, kann „Registrierter Kunde“ auch „Reservieren“ ausführen. Zusätzlich kann „Registrierter Kunde“ aber auch „Kaufen“ ausführen, was „Gast-Kunde“ nicht kann.

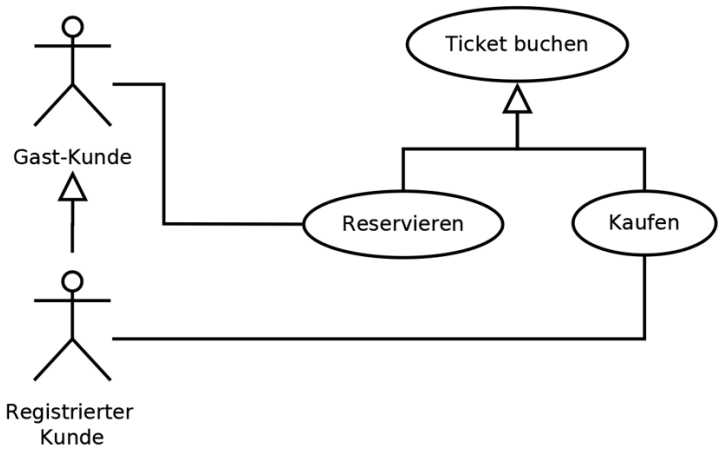


Abb.6 Darstellung Generalisierung und Spezialisierung von Anwendungsfällen und Akteuren

3.2.4 Include

Ein Anwendungsfall kann einen anderen Anwendungsfall einbinden. Dies wird über „include“ gekennzeichnet. Der eingebundene Anwendungsfall wird immer und automatisch zusammen mit dem ursprünglichen Anwendungsfall ausgeführt. (Werden neue Messdaten eingegeben, werden die Daten automatisch analysiert)

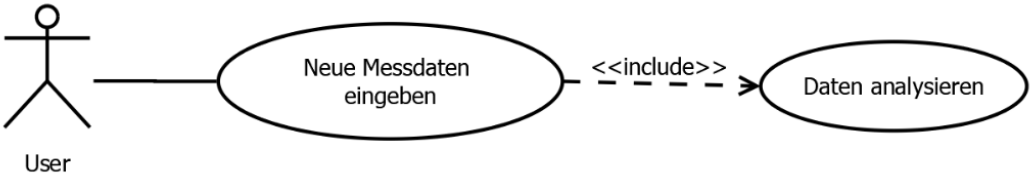


Abb.7 Darstellung eines Einbinden eines Anwendungsfall durch include

Dies ist praktisch, wenn zum Beispiel noch weitere Akteure an dem Inkludierten Use-Case beteiligt sind oder der Use-Case für Generalisierung/Spezialisierung verwendet werden soll. Include kann aber auch einfach nur dazu verwendet werden, Zusammenhänge zu modellieren und die Anwendung genauer zu beschreiben.

Wichtig: Auf die Pfeilrichtung achten! Der Pfeil zeigt auf den eingebundenen Anwendungsfall.

3.2.5 Extend

Ein Anwendungsfall kann einen anderen Anwendungsfall erweitern. Dies wird über „extend“ gekennzeichnet. Der erweiternde Anwendungsfall wird nur unter einer bestimmten Bedingung mit dem erweiterten Anwendungsfall ausgeführt.

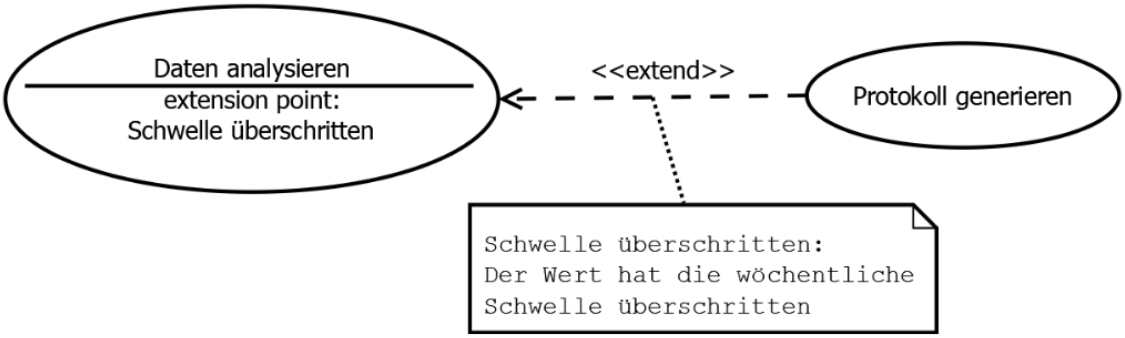


Abb.8 Darstellung einer Erweiterung eines Anwendungsfalls durch extend

Diese Darstellung zeigt, dass der Anwendungsfall "Protokoll generieren" nur dann ausgeführt wird, wenn während der Ausführung des Anwendungsfalls "Daten analysieren" die wöchentliche Schwelle für einen bestimmten Wert überschritten wird. Andernfalls wird der Anwendungsfall "Protokoll generieren" übersprungen, und die Ausführung von "Daten analysieren" wird fortgesetzt.

Die Bedingung wird als „Extension Point“ bezeichnet und kann über Notizen detaillierter beschrieben werden.

Wichtig: Auf die Pfeilrichtung achten! Der Pfeil zeigt auf den erweiterten Anwendungsfall.

3.2.6 Syntax Zusammenfassung

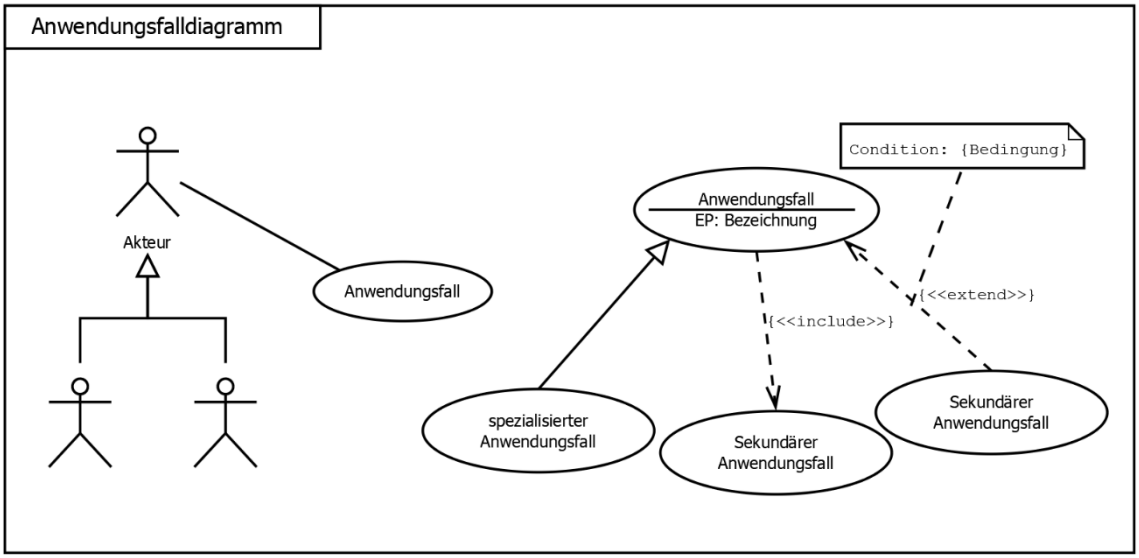


Abb.9 Darstellung Syntax Zusammenfassung

3.3 Übungsaufgabe

3.3.1 Aufgabenstellung

Erstellen Sie zu folgendem Sachverhalt ein Anwendungsfall-Diagramm:

- Alle Benutzer des Forums können Beiträge lesen.
- Unregistrierte Benutzer können sich registrieren.
- Registrierte Benutzer können sich einloggen und Beiträge verfassen.
- Zum Verfassen von Beiträgen müssen sie sich einloggen, wenn sie nicht bereits eingeloggt sind.

3.3.2 Lösung

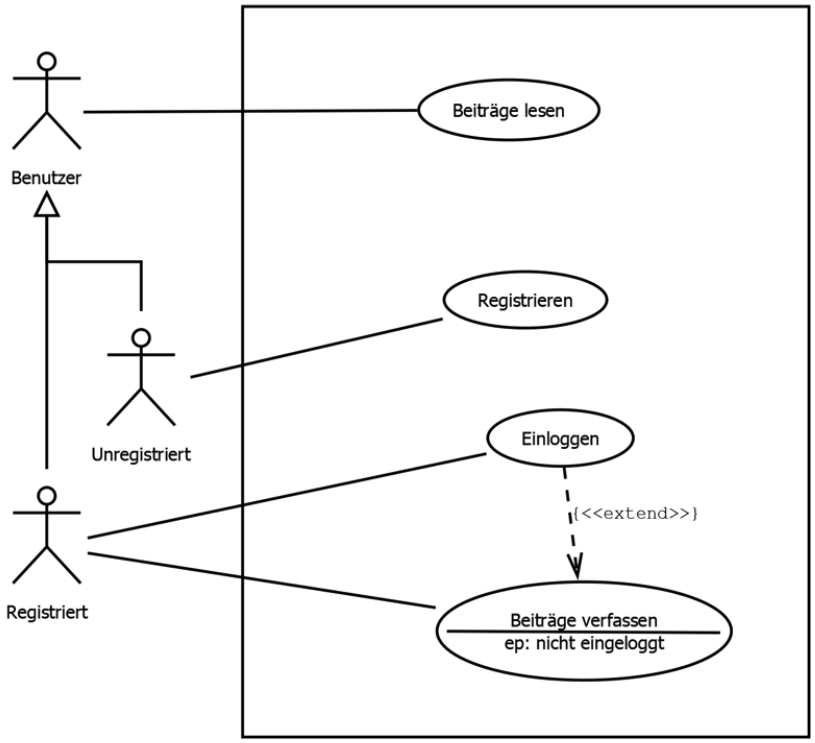


Abb.10 Darstellung Lösung zur Übungsaufgabe 3.3